

AMDP 練習 9 (期末整合) : 分析型報表用 AMDP + CDS Table Function 呈現

Lecture

這是這門課的收斂題，對照 REST 課程 rs09 期末整合的角色——不教新語法，而是把 am01~am08 學過的每個技巧，組裝成一支完整的分析型報表：

技巧	出自哪一題	這題怎麼用
AMDP Method 骨架、Client 過濾	am01	SQLScript 本體不依賴 Open SQL 的自動 Client 處理， <code>route_agg/carrier_agg</code> 兩個 CTE 都是靠 <code>mandt</code> 欄位手動串接
WITH CTE、多表 JOIN、GROUP BY 聚合	am04	兩層 CTE：先依航線 (<code>carrid+connid</code>) 彙總，再往上捲一層依 <code>carrid</code> 彙總成公司層級
CAST 做逐筆/聚合運算	am08	營收計算 <code>CAST(price * seatsocc AS DECIMAL(...))</code> ，跟載客率 <code>CAST(occ * 100.0 / max AS DECIMAL(...))</code> 都直接搬進 SQLScript
CDS Table Function、 <code>abap.clnt</code> 欄位、BY <code>DATABASE FUNCTION</code>	am07	整個 <code>ZTF_AM09_CARRIER_STATS</code> 就是把這題的統計邏輯包成一個可以直接 <code>SELECT ... FROM</code> 的 CDS Entity
Functional ALV (<code>cl_salv_table</code>)	am02 / OOP op11	呈現層只做這件事：Open SQL 查 Table Function → <code>cl_salv_table</code> 顯示，不碰任何業務邏輯

「**ABAP 端只做呈現層**」是這題最重要的設計原則：打開 `ZR_AM09_CAPSTONE` 會發現裡面完全沒有 `LOOP`、沒有任何計算，只有一句 `SELECT ... FROM ztf_am09_carrier_stats` 加上 `cl_salv_table` 顯示——所有的取數、篩選、兩層聚合、營收計算、載客率計算，全部都在 `ZCL_AM09_CARRIER_STATS` 這一個 AMDP 方法裡的 SQLScript 完成。這正是 Code-to-Data 的完整體現：業務邏輯在資料庫端，ABAP 端只負責「拿結果、給使用者看」。

學習目標

- 能整合 am01~am08 教過的技巧，設計一支「聚合層級比單一 AMDP Method 更高一層」的統計報表（這題是「依航線彙總」再捲成「依公司彙總」的兩層聚合）
- 理解「呈現層跟業務邏輯完全分離」在 AMDP+CDS Table Function 架構下長什麼樣子：ABAP 端最終只剩 `SELECT+ALV`，沒有一行計算邏輯
- 能自己設計驗證方法：這題沒有寫新的業務邏輯，而是把既有題目 (`am04/am08`) 已經驗證過的數字，拿來跟這題的彙總結果交叉核對，確認「彙總更高一層」沒有破壞底層數字的正確性

事前準備

ADT 端物件已由課程準備好 (`$TMP`)：

- **ZTF_AM09_CARRIER_STATS**——CDS Table Function，依 **carrid** 彙總：航線數、總航班數、總已訂位/總座位數、載客率、總營收
- **ZCL_AM09_CARRIER_STATS**——實作，兩層 CTE (**route_agg** 依航線彙總 → **carrier_agg** 再依公司彙總)
- **ZR_AM09_CAPSTONE**——期末報表程式，純 Open SQL 查詢 + **cl_salv_table** 顯示，沒有任何業務邏輯

題目需求 (對照已建好的答案物件)

1. **ZTF_AM09_CARRIER_STATS**：

```
``abap @EndUserText.label: 'AM09 capstone: carrier-level stats table function' define
table function ZTF_AM09_CARRIER_STATS returns { mandt : abap.clnt; carrid : s_carr_id;
carrname : s_carrname; route_cnt : abap.int4; total_flights : abap.int4; total_seats_occ
: abap.int4; total_seats_max : abap.int4; load_pct : abap.dec(5,1); total_revenue :
abap.dec(15,2); } implemented by method zcl_am09_carrier_stats=>get_data; ``
```

1. **ZCL_AM09_CARRIER_STATS** (兩層 CTE，BY DATABASE FUNCTION，承 am07)：

```
``abap METHOD get_data BY DATABASE FUNCTION FOR HDB LANGUAGE SQLSCRIPT OPTIONS READ-ONLY
USING scarr sflight.
```

```
RETURN WITH route_agg AS ( SELECT f.mandt, f.carrid, f.connid, COUNT(*) AS flight_cnt, SUM(f.seatsocc) AS
seats_occ, SUM(f.seatsmax) AS seats_max, SUM(CAST(f.price * f.seatsocc AS DECIMAL(15,2))) AS route_revenue
FROM sflight AS f GROUP BY f.mandt, f.carrid, f.connid ), carrier_agg AS ( SELECT mandt, carrid, COUNT(*) AS
route_cnt, SUM(flight_cnt) AS total_flights, SUM(seats_occ) AS total_seats_occ, SUM(seats_max) AS
total_seats_max, SUM(route_revenue) AS total_revenue FROM route_agg GROUP BY mandt, carrid ) SELECT
ca.mandt, ca.carrid, c.carrname, ca.route_cnt, ca.total_flights, ca.total_seats_occ, ca.total_seats_max,
CAST(ca.total_seats_occ * 100.0 / ca.total_seats_max AS DECIMAL(5,1)) AS load_pct, ca.total_revenue FROM
carrier_agg AS ca JOIN scarr AS c ON c.mandt = ca.mandt AND c.carrid = ca.carrid;
```

```
ENDMETHOD. ``
```

route_agg 這一層，本質上就是 **am04** 的 **ZCL_AM04_ROUTE_LOAD** 在算的東西 (依航線彙總)，只是這裡多存了一欄 **route_revenue** (承 **am08** 的營收計算)；**carrier_agg** 是新加的第二層，把 **route_agg** 的每一列 (每條航線) 再依 **carrid** 捲總——這就是「彙總層級比單一 AMDP Method 更高一層」的具體做法：CTE 可以疊很多層，每一層都是對上一層的結果再做一次聚合。

1. **ZR_AM09_CAPSTONE** (呈現層，沒有任何業務邏輯)：

```
``abap REPORT zr_am09_capstone.
```

```
START-OF-SELECTION.
```

```
SELECT carrid, carrname, route_cnt, total_flights, total_seats_occ, total_seats_max, load_pct, total_revenue
FROM ztf_am09_carrier_stats ORDER BY carrid INTO TABLE @DATA(lt_stats).
```

```
cl_salv_table=>factory( IMPORTING r_salv_table = DATA(lo_alv) CHANGING t_table = lt_stats ).
```

```
DATA(lo_columns) = lo_alv->get_columns( ). lo_columns->set_optimize( abap_true ).
```

```
lo_columns->get_column( 'CARRID' )->set_medium_text( 'Carrier' ). lo_columns->get_column( 'CARRNAME' )-
>set_medium_text( 'Airline' ). lo_columns->get_column( 'ROUTE_CNT' )->set_medium_text( 'Routes' ).
lo_columns->get_column( 'TOTAL_FLIGHTS' )->set_medium_text( 'Flights' ). lo_columns->get_column(
'TOTAL_SEATS_OCC' )->set_medium_text( 'Seats Occ.' ). lo_columns->get_column( 'TOTAL_SEATS_MAX' )-
>set_medium_text( 'Seats Max' ). lo_columns->get_column( 'LOAD_PCT' )->set_medium_text( 'Load %' ).
lo_columns->get_column( 'TOTAL_REVENUE' )->set_medium_text( 'Revenue' ).
```

```
lo_alv->display(). ``
```

實測踩坑：一開始沒有手動設欄位標題，**SAP GUI** 實際執行後發現除了 **CARRID / CARRNAME** 兩欄有正常標題 (**Carrier / Airline**) 之外，其餘欄位 (**ROUTE_CNT / TOTAL_FLIGHTS / ...**) 標題全部空白：

cl_salv_table 的欄位標題預設是靠 RTTI (執行期型別反射) 去讀取每個欄位背後對應的 Data Element 說明文字自動生成——**carrid/carrname** 因為在 CDS 裡引用的是 **s_carr_id/s_carrname** 這兩個既有 Data Element (本身就帶著中/英文欄位說明)，所以標題正常帶出來；但 **ZTF_AM09_CARRIER_STATS** 的 **route_cnt/total_flights/...** 這幾個彙總欄位，**returns** 結構裡直接用 **abap.int4/abap.dec(...)** 這種內建型別宣告 (沒有另外建 Data Element)，完全沒有欄位說明文字可以讓 RTTI 抓，所以標題是空的。修正方式：用 **get_columns()->get_column('<欄位名>')->set_medium_text('<標題>')** 手動幫每一欄補標題——這跟 **am06** 手動組 **IT_FIELDCAT** 遇到的根本問題是同一件事 (欄位沒有對應 Data Element、ALV 就沒有現成的標題可用)，只是這裡用的是 **cl_salv_table** 的欄位物件 API，不是經典 ALV 的欄位目錄表格。

預期輸出 (實測畫面，8 家有航班紀錄的航空公司)

AA	American Airlines	2	25	5,712	8,910	64.1	2,415,833.28
AZ	Alitalia	4	52	12,361	18,915	65.3	11,793,230.00
DL	Delta Airlines	3	39	9,113	13,845	65.8	4,353,013.86
JL	Japan Airlines	2	26	7,464	11,180	66.7	7,921,991.04
LH	Lufthansa	5	76	17,112	25,450	67.2	8,845,505.99
QF	Qantas Airways	2	26	4,996	7,670	65.1	3,940,045.44
SQ	Singapore Airlines	4	52	12,292	18,330	67.0	22,550,202.70
UA	United Airlines	4	60	16,440	23,925	68.7	12,081,508.96
Total carriers: 8							

這份結果直接跟 **am04**、**am08** 已經驗證過的數字交叉核對：**LH** 這一行 **route_cnt=5**、**total_flights=76**、**total_seats_occ=17,112**、**total_seats_max=25,450**、**total_revenue=8,845,505.99**——

- **route_cnt=5**、**total_flights=76**：跟 **am04** 實測畫面裡 **LH** 五條航線 (**0400/0401/0402/2402/2407**) 的 **flight_cnt** 分別是 **16+15+15+15+15=76** 完全對得上
- **total_seats_occ=17,112**：**am04** 五條航線的 **seats_occ** 分別是 **3220+2716+5148+4768+1260=17112**，完全對得上
- **total_seats_max=25,450**：**am04** 五條航線的 **seats_max** 分別是 **5350+3900+7125+7125+1950=25450**，完全對得上
- **total_revenue=8,845,505.99**：跟 **am08** 實測的 **LH** 總營收 (**ZR_AM08_DEMO** 用 **p_carrid=LH** 跑出來的數字) 一字不差

只有 **18** 家航空公司裡的 **8** 家出現在結果裡——這是因為這題的 CTE 是從 **SFLIGHT** 出發 (**INNER JOIN** 到 **SCARR**)，沒有航班紀錄的公司自然不會出現，跟 **am04/am07** 觀察到的現象一致。

團隊實務備註

- 這題沒有寫任何一行「新的」業務邏輯，**route_agg** 幾乎是把 **am04** 的邏輯原封不動搬過來，**carrier_agg** 才是真的新加的一層——這是刻意的：期末整合題的重點是「組裝」跟「驗證組裝後結果還是對的」，不是「示範更多新語法」，如果為了這題硬塞新技巧，反而模糊了「整合」這個目的
- 驗證一支彙總報表最可靠的方法，是找到已經驗證過的更細顆粒度資料，手算捲總、跟報表結果對答案——這題沒有另外寫 ABAP Unit 測試或額外的程式碼來驗證，而是直接拿 am04（航線層級）、am08（單一航空公司營收）已經人工驗證過的數字，加總起來對照這題的公司層級結果，三個獨立驗證過的資料點全部吻合，比起「肉眼看數字合不合理」更有說服力
- **ZR_AM09_CAPSTONE** 完全沒有 **iv_mandt/iv_carrid** 這類 **IMPORTING** 參數，因為它是純 **Open SQL 查詢 Table Function**、不是直接呼叫 **AMDP Method**——如果之後想加篩選條件（例如只看某幾家公司），有兩種做法：(1) 在 **SELECT ... FROM ztf_am09_carrier_stats** 後面加 Open SQL 的 **WHERE carrid IN ...**（篩選在 CDS 框架層做，AMDP 已經算完全部資料後才篩），(2) 幫 Table Function 加 **with parameters**、AMDP 方法對應加 **IMPORTING** 參數，把篩選條件下推進 SQLScript 本體（篩選跟聚合一起在資料庫端做）——兩者效能可能有差異，資料量大時第二種通常更好，這是 am08 教過的「何時該下推」判斷的延伸應用

課程回顧

到這裡，AMDP / SQLScript / Code-to-Data 這門課的 am01~am09 就結束了。整理一下九題各自解決的問題：

#	主題	核心收穫
am01	架構總覽	AMDP 骨架、 AMDP 不會自動處理 Client
am02	Signature 規則	多個 EXPORTING Table、ALV 呈現
am03	變數與流程控制	DECLARE / CURSOR / FOR / IF ，宣告式 vs 命令式
am04	JOIN / CTE / 聚合	WITH CTE、JOIN 條件要不要寫 MANDT （跟 Open SQL 相反）
am05	錯誤處理	SIGNAL / CX_AMDP_ERROR ，錯誤訊息很技術性
am06	除錯與資料預覽	AMDP Debugger（手動操作）、Data Preview
am07	CDS Table Function	BY DATABASE FUNCTION 、 abap.clnt 欄位、透過 Open SQL 查詢反而自動過濾 Client
am08	Code-to-Data 實戰改寫	下推不是無條件更快，第一次呼叫有編譯成本
am09	期末整合	組裝 + 交叉驗證，呈現層跟業務邏輯完全分離

答案

見 **ztf_am09_carrier_stats.ddls.asddls**、**zcl_am09_carrier_stats.clas.abap**、**zr_am09_capstone.prog.abap**（SAP 端物件 **ZTF_AM09_CARRIER_STATS** / **ZCL_AM09_CARRIER_STATS** / **ZR_AM09_CAPSTONE**，套件 **\$TMP**）。